

NETRA

Network-wide Trajectory & Recognition Analytics

Single-camera ANPR is a solved commodity. The unsolved part of PS 127 is what happens *between* cameras: plate reads from real CCTV are noisy, partial and sometimes wrong, and a city-scale system that treats OCR output as ground truth produces confident nonsense. **NETRA treats every plate read as a hypothesis, not a fact** — and resolves vehicle identity across cameras by fusing plate similarity, appearance, and whether the journey is physically possible at all.

PROBLEM STATEMENT	MILESTONES	FIRST DEMOABLE BUILD	TARGET HARDWARE
PS 127	12	Milestone 6	One laptop

DEMO CAMERAS

3 – 5 feeds

SECTION 01

The read on PS 127

Before any architecture, the problem restated in terms we can actually build and defend in front of judges.

Problem definition

A city operates hundreds of traffic cameras that each see a slice of the road network. Every camera is an island: it can count what passes and, at best, read a plate. Nobody can answer the questions that actually matter for traffic management — *where did this vehicle come from, where is it going, how long did that journey take, and which corridor is choking right now* — because no layer stitches the islands together into a single view of a vehicle moving through the city.

The gap we're attacking

The gap is not detection and it is not OCR. Both are mature. The gap is **identity continuity under uncertainty**. A vehicle leaving Camera A and arriving at Camera D is the same physical object, but the system sees two unrelated blobs, two independent plate strings that may differ by two

characters, and nothing linking them. Naive string matching on plates fails in both directions: it misses real matches (OCR read `MH12DE1433` as `MH120E1A33`) and it invents false ones (two different vehicles both degrade to the same garbage read).

Our angle — the thing that makes this a project and not a tutorial

A three-signal, confidence-scored identity resolver. Every cross-camera link is scored on:

- 1. **Plate similarity** — edit distance weighted by a real OCR *confusion matrix* (`0↔0`, `1↔I`, `8↔B`, `5↔S`, `2↔Z`, `6↔G`), so visually-confusable substitutions cost less than arbitrary ones, plus an Indian plate-format grammar that both validates and *repairs* reads (position 0–1 must be letters, so an `0` there is almost certainly an `0`).
- 2. **Appearance** — vehicle class, dominant colour and size ratio as a cheap first pass; a learned re-ID embedding as the upgrade path.
- 3. **Physical plausibility** — a camera graph with real road distances. Every candidate link implies a travel speed. If that speed is negative or above a corridor ceiling, the link is *rejected regardless of how well the plates matched*.

WHY JUDGES WILL CARE

The rejection path is the demo's credibility moment. Showing a link the system *refused* — "plate similarity 0.94, but implied speed 214 km/h over a 3.1 km link, rejected" — proves there is real reasoning under the dashboard, not a lookup table. Almost no competing team will show a negative result on purpose.

Key features we commit to

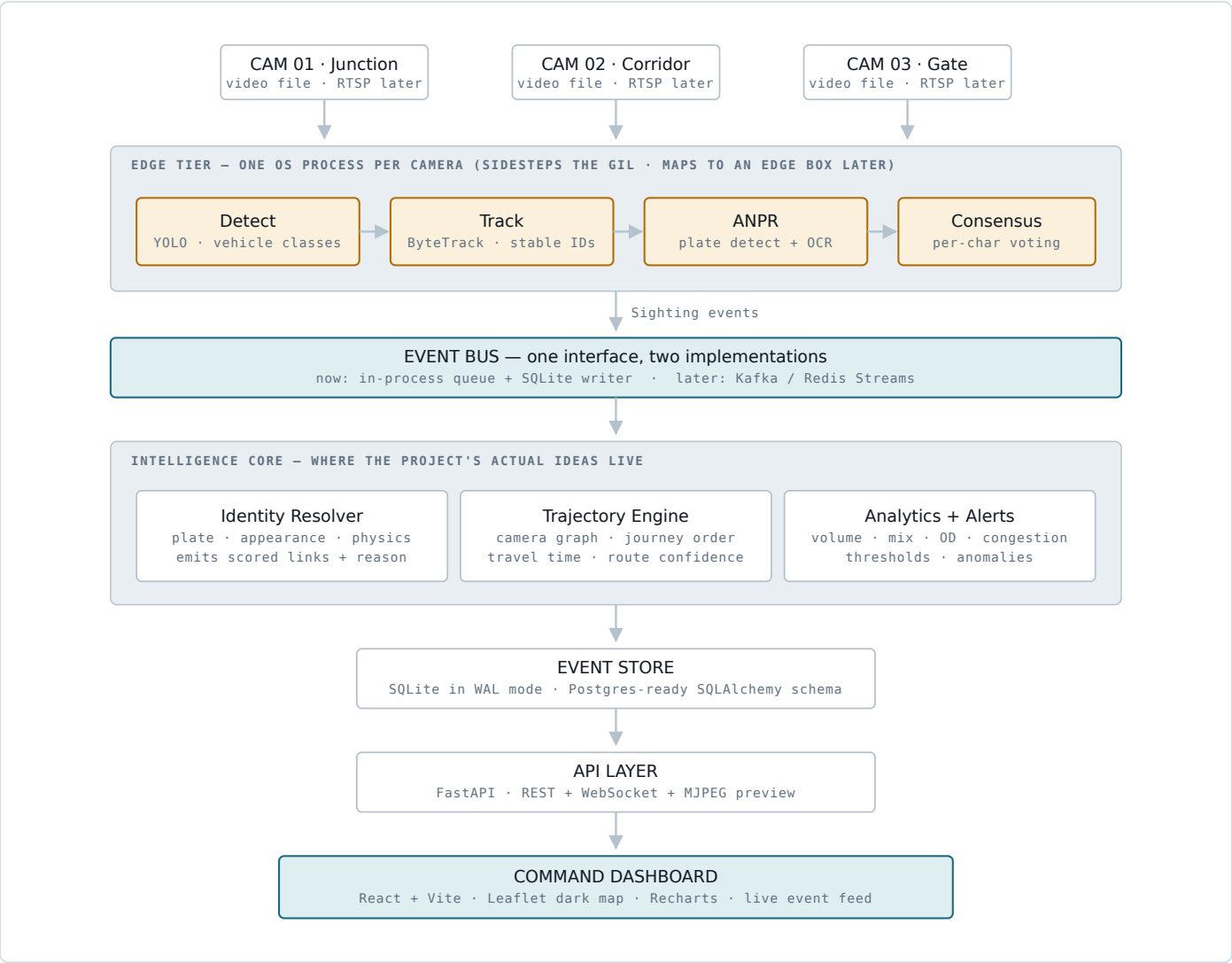
CAPABILITY	WHAT IT ACTUALLY DOES	PRIORITY
Camera manager	Config-driven camera registry: ID, name, lat/lon, heading, source URI, status. One process per camera.	MUST
Vehicle detection	Car / motorcycle / bus / truck / auto-rickshaw class detection per frame, with confidence.	MUST
In-camera tracking	Stable track IDs across frames, entry/exit timestamps, direction of travel, line-crossing counts.	MUST
ANPR + consensus	Plate crop → OCR per frame → per-character confidence-weighted vote across the whole track → one consensus plate.	MUST
Identity resolver	Three-signal scoring above; emits links with an explicit confidence and a human-readable reason.	MUST
Trajectory engine	Ordered journey reconstruction over the camera graph, with per-hop travel time, distance and route confidence.	MUST
Analytics	Volume, vehicle mix, corridor travel time, congestion index, OD matrix, peak periods, hotspots.	MUST

CAPABILITY	WHAT IT ACTUALLY DOES	PRIORITY
Command dashboard	Live camera grid, dark city map with animated trajectories, plate search, event feed, charts, alerts.	MUST
Alerts	Congestion threshold breach, corridor over-speed, watchlist plate hit, anomalous route.	NICE
Learned re-ID	Appearance embedding to link vehicles whose plates were never readable.	NICE

SECTION 02

Architecture

One rule shapes everything below: the prototype's structure must be the production structure with cheaper parts substituted in. If a scaling claim requires rewriting the system, it isn't a scaling claim.



Data flows one way. Nothing above the bus knows anything about the dashboard; nothing below it knows anything about video decoding. That separation is what lets us claim the scaling story honestly.

The four decisions that actually matter

1 · One OS process per camera, not one thread per camera

PROBLEM	Python's GIL means threaded camera workers serialise on inference and we get one camera's worth of throughput no matter how many feeds we open.
SIMPLEST FIX	A supervisor that spawns N independent worker processes from <code>cameras.yaml</code> . Each owns its own model instance and video decoder.
UPGRADE PATH	The same worker binary runs on a Jetson/edge box next to the real camera. Zero code change — only the source URI and the bus implementation move.

2 · An event bus interface, even though the prototype doesn't need one

PROBLEM	Every hackathon scalability slide claims "we'd add Kafka" and every judge knows it means a rewrite.
SIMPLEST FIX	Roughly forty lines: an abstract <code>EventBus</code> with <code>publish()</code> / <code>subscribe()</code> , and a <code>LocalBus</code> that writes to SQLite. That's the entire cost.
UPGRADE PATH	<code>KafkaBus</code> is a second implementation of the same interface. The claim becomes demonstrable rather than aspirational — and that is worth real marks.

3 · Plate consensus happens in the worker, not in the database

PROBLEM	A vehicle is visible for 40–90 frames. Writing 90 independent OCR guesses to the store and reconciling later is expensive and loses per-character confidence.
SIMPLEST FIX	The worker buffers reads for the lifetime of a track, then on track-exit emits <i>one</i> sighting: consensus plate, per-character confidence vector, best crop, entry/exit times, class, colour.
UPGRADE PATH	Nothing needed. This is also how commercial ANPR works — it's the correct design, and it drops event volume by ~50×.

4 · SQLite, not PostgreSQL

PROBLEM	Postgres costs an afternoon of Docker plumbing on demo day and buys us nothing at three cameras.
SIMPLEST FIX	SQLite in WAL mode behind SQLAlchemy. Handles thousands of writes per second; zero setup; the demo runs after a <code>git clone</code> .
UPGRADE PATH	Identical ORM models. Change one connection string to move to Postgres + TimescaleDB. Say exactly that on the slide.

SECTION 03

Tech stack, and what we rejected

Every row was chosen for reliability on one laptop under demo conditions. Where a component is a real risk, the fallback is named up front rather than discovered at 2 a.m.

LAYER	CHOICE	WHY THIS	REJECTED / FALLBACK
Detection	ultralytics · yolo11n → yolo26n	COCO already has car / motorcycle / bus / truck. Start on YOLO11 (maximum stability, huge community), swap to the newer YOLO26 via a config value once the pipeline is green — it's NMS-free and faster on CPU.	Training a custom detector. Zero demo value, days of cost.
Tracking	ByteTrack (built into ultralytics)	<code>model.track(persist=True)</code> — one argument. Implementation cost is effectively nil and it is a genuinely strong tracker.	Hand-rolled DeepSORT. BoT-SORT+ReID is the upgrade if we want embeddings for free.
ANPR	fast-alpr (ONNX, MIT)	Bundles a plate detector plus a global plate-OCR model, runs on CPU through ONNX Runtime, installs cleanly on Apple Silicon. Plate detect + read in about five lines.	PaddleOCR — better raw OCR, but the paddlepaddle wheel is a known trap on Apple Silicon. EasyOCR is the tested fallback.
Plate grammar	Indian format regex + confusion matrix	Ours to write, ~60 lines, and it is the single highest-leverage accuracy win in the project.	Trusting raw OCR strings. Guaranteed to embarrass us live.
Store	SQLite (WAL) + SQLAlchemy	Zero setup, survives demo-day networking, schema-identical to Postgres.	Postgres in Docker. Costs an afternoon, buys nothing at this scale.
API	FastAPI + uvicorn	REST, WebSocket and MJPEG streaming from one process. Auto-generated <code>/docs</code> is a free credibility slide.	Flask (no async / WS story), gRPC (nothing gains from it here).

LAYER	CHOICE	WHY THIS	REJECTED / FALLBACK
Frontend	Vite + React + TypeScript + Tailwind	Fastest path to a polished dark UI. No SSR complexity we'd never use.	Next.js — server rendering is pure overhead for a local dashboard.
Map	react-leaflet + CARTO dark basemap	Genuinely free, no API key , and dark tiles out of the box — which is most of the command-centre look.	Mapbox/Google — API keys, quotas, and a billing card is the last thing you want on demo day.
Charts	Recharts	React-native API, good defaults, quick to theme.	D3 by hand. Hours spent on axes nobody will look at closely.
Orchestration	Python multiprocessing + one Makefile	<code>make demo</code> starts everything. That's the whole DevOps story.	Docker Compose, Kubernetes, message brokers, cloud. Explicitly out of scope.

HARDWARE NOTE – APPLE SILICON

On a MacBook Pro, YOLO runs on `device="mps"` and ONNX Runtime uses the CPU. Expect roughly 15–25 FPS per camera on a nano model at 720p. That is comfortably enough: we run ANPR on every n -th frame, not every frame, and the worker paces playback to wall-clock time anyway.

SECTION 04

Data strategy

This section is the highest-risk part of the project and it is not a coding problem. Read it before writing any code.

RISK 1 – THE ONE THAT CAN SINK THE DEMO

Public CCTV and YouTube traffic footage will not give you readable plates. Overhead municipal cameras are mounted for flow, not identification, and re-compression destroys what little plate detail survives. If plates are unreadable, the cross-camera engine has nothing to work with and the entire project collapses into a vehicle counter.

The working threshold: **a plate needs to be at least ~80 pixels wide** in the frame for OCR to be reliable. Check any candidate clip against that number *before* building around it.

The fix: shoot your own three-camera dataset

Two hours with your team and three phones is the highest-leverage work in this entire project. Pick a campus gate, a parking exit, or a low-speed service road. Place three phones at points A, B and C

along a route, 1080p or 4K, angled so plates are large and roughly frontal. Then have teammates drive or ride the same six to eight vehicles past A → B → C in a recorded order.

What that single afternoon buys you:

- **Readable plates**, which is non-negotiable.
- **Genuine cross-camera journeys** — not simulated ones. The core claim of the project becomes real.
- **Ground truth**. You know which vehicle went where and when, so you can compute actual precision and recall.
- **A slide no other team has**: "we collected our own multi-camera evaluation set."

Practical notes: start all three recordings against a synced clock (film one phone showing the time on each camera), keep cameras static, shoot in good light, and log the ground-truth pass order in a spreadsheet as you go.

RISK 2 – AND WHY WE'RE COVERED ON MEASUREMENT

SIH judges ask "how do you know it works?" Almost nobody can answer. With the ground-truth log above, we build a 30-line evaluation script and answer with numbers: OCR character accuracy on N plate crops, and cross-camera association precision/recall on N known journeys. Budget one hour. It is worth more than any dashboard feature.

Supplementary public data

SOURCE	USE IT FOR	CAVEAT
Roboflow Universe — Indian number plates	Fine-tuning the plate detector on Indian plate geometry if the bundled one underperforms.	Quality varies per dataset; check a sample before training.
UA-DETRAC / open traffic clips	Detection and tracking robustness; busy scenes for the analytics numbers.	Plates generally unreadable — do not use these for ANPR.
CityFlow / AI City Challenge	The canonical multi-camera vehicle re-ID benchmark; a strong citation for the presentation.	Requires a signed data request. Apply now if at all, but never let it block us.
Dashcam / toll-plaza footage	High-quality plate crops for the OCR bake-off harness.	Single-camera only — no journey structure.

The rule: own footage is the demo spine; public data is supporting evidence. Never let a dataset download block a milestone.

The 12-milestone roadmap

Reordered from the original 16. Three structural changes: data collection moves to the very front because it blocks everything; the database schema lands before ANPR because every stage writes to it; and the dashboard starts at Milestone 6 as a thin vertical slice rather than at the end.

THE INSURANCE POLICY

From Milestone 6 onward, the system is always demoable. Every milestone after that adds capability to a running system rather than moving toward a first run. If we run out of time at any point past M6, we still have something to put in front of judges. This is the single most important scheduling decision in the plan.

01 Skeleton, config & the video pipeline MUST ~2 h

Repo structure, dependency install, YAML config loader, logging, and a `VideoSource` abstraction that reads a file today and an RTSP URL later, paced to wall-clock FPS so a 30-fps file behaves like a live camera.

ACCEPTANCE TEST

Run the preview CLI against CAM01. A window plays the clip at real-time speed with a frame counter and live FPS overlay; changing `cameras.yaml` changes what plays, with no code edit.

02 Vehicle detection MUST ~2 h

YOLO wrapped behind a `Detector` class with the model name in config. Filter to vehicle classes, draw boxes with class and confidence, report inference milliseconds per frame.

ACCEPTANCE TEST

Vehicles are boxed and labelled by type; the FPS overlay shows a workable rate; swapping the model name in config changes the model with no code edit.

03 Tracking & line counting MUST ~3 h

ByteTrack for persistent IDs. A `Track` object accumulating first-seen, last-seen, class votes, and the box history. A configurable virtual counting line per camera with direction of crossing.

ACCEPTANCE TEST

A single vehicle keeps one ID from entering to leaving the frame, including through a brief occlusion. The counter increments once per vehicle per crossing – never twice.

04 Event store & the bus interface MUST ~3 h

SQLAlchemy models for `cameras`, `sightings`, `plate_reads`, `vehicle_identities`, `links`, `journeys`, `alerts`. SQLite in WAL mode. The `EventBus` interface plus its local implementation. Tracks now persist as sightings on exit.

ACCEPTANCE TEST

Process a clip, then query the DB: one row per vehicle that crossed, with correct camera ID, timestamps, class and track ID. Row count matches the on-screen counter.

05 ANPR with temporal consensus MUST ~6 h · the hard one

Plate detection and OCR per track. A small bake-off harness comparing fast-alpr against EasyOCR on ~30 hand-cropped plates, so the choice is measured rather than assumed. Then the part that matters: per-character confidence-weighted voting across every read in a track, Indian plate-format validation, and format-guided repair.

ACCEPTANCE TEST

For a vehicle whose plate you know, the consensus string is correct even though several individual frames read it wrong. The bake-off script prints a character-accuracy table for both engines.

06 Multi-camera orchestration + first vertical slice MUST ~5 h

A supervisor spawning one worker process per camera. FastAPI with a handful of endpoints, a WebSocket event feed, and MJPEG preview per camera. A minimal React dashboard: camera tiles, KPI row, live ANPR feed. **This is the first build you could show someone.**

ACCEPTANCE TEST

Three cameras process simultaneously; the browser shows three live annotated tiles and plate reads appearing in the feed in real time. Killing one worker leaves the other two running.

Cross-camera identity resolution MUST

~6 h · the centrepiece

The three-signal resolver: confusion-matrix-weighted plate similarity, appearance features, and the travel-time plausibility gate over the camera graph. Every link carries a confidence and a human-readable reason. Plus the evaluation script against your ground-truth log.

ACCEPTANCE TEST

A vehicle recorded passing CAM01 then CAM02 is linked to one identity, despite the two plate reads differing by a character. A deliberately implausible pair (same plate, 8 seconds apart, 3 km apart) is rejected with the reason logged. The eval script prints precision and recall.

Trajectory engine MUST

~4 h

The camera graph in config: nodes with coordinates, edges with real road distance and a speed ceiling. Chain resolved links into ordered journeys with per-hop distance, duration, implied average speed, and an overall route confidence.

ACCEPTANCE TEST

Query a plate via the API and get back an ordered journey – camera, timestamp, hop distance, hop duration, confidence – plus journey totals matching what you physically recorded.

Analytics & alerts MUST

~4 h

Volume time series, vehicle-type mix, per-corridor travel-time distribution, a congestion index derived from travel time versus free-flow baseline, an origin-destination matrix across camera pairs, peak-period detection, and hotspot ranking. Alert rules on top: congestion breach, corridor over-speed, watchlist plate hit.

ACCEPTANCE TEST

The analytics endpoints return populated, sane numbers from real processed data. Lowering an alert threshold in config makes a matching alert fire and appear on the WebSocket feed.

10

The command dashboard

MUST

~8 h

The full UI: dark Leaflet map with camera nodes and animated trajectory playback, live camera grid, KPI header, plate search with a journey timeline, analytics charts, and an alerts panel. Every displayed number sourced from the API — nothing hardcoded, nothing invented.

ACCEPTANCE TEST

Type a plate into search and watch its path animate across the map with per-hop confidence shown. Every KPI on screen can be traced to a real API response.

11

Demo hardening

MUST

~4 h

A seed script that pre-populates the DB with a full processed run so analytics look rich the instant the dashboard opens — then run the live pipeline on top for the "watch it happen" moment. One-command startup. Graceful handling of a dead worker or an empty result. A known-good fallback clip. Rehearse the run three times.

ACCEPTANCE TEST

On a freshly rebooted laptop with no network, one command brings the whole system up in under 60 seconds with populated analytics. Unplug a camera worker mid-demo and the dashboard degrades gracefully instead of erroring.

12

Presentation package

MUST

~4 h

Architecture diagram, the prototype→production mapping table, your measured accuracy numbers, an honest limitations slide, future scope, and a timed demo script with assigned speaking roles.

ACCEPTANCE TEST

A teammate who did not write the code can deliver the full six-minute demo unaided, including answering "how do you handle a misread plate?"

SECTION 06

Repository structure

One repo, two apps. Module boundaries follow the architecture diagram exactly, so a judge reading the folder tree can see the design without opening a file. Highlighted paths are what Milestone 1 creates.

```
netra/  
├── README.md
```

```

├─ Makefile                                # make setup · make demo · make eval
├─ configs/
│   ├── system.yaml                       # model names, thresholds, paths, device
│   ├── cameras.yaml                     # camera registry: id, name, lat/lon, heading, source
│   └─ city_graph.yaml                   # nodes + edges with road distance & speed ceiling
├─ data/
│   ├── videos/                          # your recorded camera footage
│   ├── models/                          # downloaded weights (gitignored)
│   ├── groundtruth/                     # the pass-order log from your shoot
│   └─ netra.db                          # SQLite (gitignored)
├─ backend/
│   ├── requirements.txt
│   ├── app/
│   │   ├── core/                        # config loader, logging, event schemas, EventBus
│   │   ├── vision/                     # VideoSource, Detector, Tracker, ANPR, plate grammar
│   │   ├── pipeline/                   # camera_worker.py, supervisor.py
│   │   ├── identity/                   # consensus, similarity, appearance, resolver
│   │   ├── trajectory/                 # camera graph, journey reconstruction
│   │   ├── analytics/                  # metrics, congestion, OD matrix, alert rules
│   │   ├── store/                      # SQLAlchemy models, session, repositories
│   │   ├── api/                        # FastAPI app, routers, websocket, mjpeg
│   │   └─ cli.py                       # preview · process · serve · seed
│   ├── scripts/                         # ocr_bakeoff.py, eval_association.py, seed_demo.py
│   └─ tests/
├─ frontend/
│   ├── package.json
│   └─ src/
│       ├── components/                 # CameraGrid, CityMap, EventFeed, KpiBar, PlateSearch...
│       ├── pages/                      # Live, Analytics, Search
│       ├── hooks/                      # useWebSocket, useApi
│       └─ lib/                         # api client, types, theme

```

Two conventions worth fixing now, because they get expensive to retrofit: **every tunable number lives in `configs/`**, **never in code** — thresholds, model names, FPS caps, alert limits — and **every module boundary in the tree matches a box in the architecture diagram**, so the slide and the codebase never drift apart.

The demo story

Design the demo now, build toward it. Six minutes, one uninterrupted narrative, no slide-flipping mid-run. Every beat below is a feature we've committed to shipping.

- | | |
|------|---|
| 0:00 | The gap. "This city has 400 cameras and 400 blind spots. Each one sees a moment. None of them sees a journey." One line, no preamble. |
| 0:30 | Open on a live system. Dashboard already populated from the seeded run — three camera tiles playing with live detection boxes, KPI row counting up, plate reads scrolling into the event feed with confidence scores. No empty state, ever. |
| 1:15 | The pipeline, in ten seconds. Point at one tile: detected → tracked → plate read → consensus across 60 frames → one sighting. Emphasise the consensus step: "we never trust a single frame." |
| 2:00 | The money shot. Ask a judge to pick a plate from the live feed. Search it. The map animates the vehicle's real path across three cameras with timestamps, hop distances and per-hop confidence. |
| 3:00 | The credibility moment. Open the rejected-links panel. "Here's a pair we refused — plate similarity 0.94, but that link implies 214 km/h over 3.1 km. Physically impossible, so we didn't assert it." This is the beat that separates you from every other team. |
| 3:45 | Why a city cares. Analytics tab: corridor travel times, congestion index over the hour, the OD matrix, hotspot ranking. Then an alert firing live in the feed. |
| 4:45 | The numbers. "On our own three-camera recorded dataset with ground truth: X% plate character accuracy, Y% association precision, Z% recall." Nobody else will have this. |
| 5:15 | Scale and honesty. The prototype→production table below. Then state the limitations before a judge has to ask them — it reads as confidence, not weakness. |

Prototype → production, stated plainly

LAYER	TODAY (PROTOTYPE)	AT CITY SCALE	DOES THE CODE CHANGE?
Ingest	Recorded files paced to real time	RTSP from municipal CCTV	No – swap the source URI
Inference	Worker process on a laptop	Same worker on an edge box or GPU node	No – same worker binary
Transport	LocalBus → SQLite	KafkaBus → stream processors	One class, same interface
Store	SQLite (WAL)	Postgres + TimescaleDB	One connection string
Intelligence	In-process modules	Horizontally scaled consumers	Stateless already
Dashboard	Local React app	Same app, hosted	No

LIMITATIONS WE STATE BEFORE WE'RE ASKED

Plate readability depends on camera placement and resolution; the prototype is evaluated on a small self-collected dataset, not at city scale; appearance re-identification degrades badly at night; association confidence is calibrated on our own data and would need re-calibration per deployment; and a real deployment carries privacy and data-retention obligations that a hackathon prototype does not address. Saying this yourself is worth more than being caught by it.

Must have · Nice to have · Don't waste time

MUST HAVE

- Your own multi-camera footage with ground truth
- Detection + tracking on 3 cameras at once
- Plate consensus across a track
- Confidence-scored cross-camera association
- The travel-time plausibility gate
- Journey reconstruction & plate search
- Measured accuracy numbers
- A dark, polished dashboard with a map
- One-command startup + seeded data

NICE TO HAVE

- Learned re-ID embeddings for unreadable plates
- Speed estimation via homography
- Watchlist / BOLO alerting
- Anomalous-route detection
- Fine-tuned Indian plate detector
- Trajectory playback scrubber
- Live RTSP from one real IP camera

DON'T WASTE TIME

- Kubernetes, Docker Compose, any cloud
- Microservices beyond the worker split
- Training a detector from scratch
- Authentication and user management
- Postgres for the prototype
- Mapbox / anything needing an API key
- Mobile app, CI/CD, unit-test coverage targets
- Real-time processing of more than ~5 feeds

What we build first

Two things start in parallel, today

1. **Team task — the camera shoot.** It needs daylight, teammates and vehicles, so it can't be done at 1 a.m. and it blocks Milestones 5, 7 and 8. Three phones, three points along one route, six to eight vehicles driven past A → B → C, synced clocks, pass order written down. Check plate width against the ~80 px rule before you leave the location.
2. **Engineering task — Milestone 1.** Repo skeleton, dependency install, config system, logging, and the `VideoSource` abstraction with a preview CLI. Deliberately unglamorous: it's the contract every later milestone plugs into, and getting the config-driven camera registry right now is what makes Milestone 6 take five hours instead of fifteen.

Milestone 1's acceptance test: `python -m app.cli preview --camera CAM01` plays your clip at wall-clock speed with a frame counter and FPS overlay, and editing `cameras.yaml` changes what plays without touching a line of code.